Trusted by Proxy: Source-Laundering Attacks in Multi-Agent LLM Systems

Anonymous authors
Paper under double-blind review

Abstract

Multi-agent LLM systems decompose user tasks across agents that read documents, summarize information, plan actions, and execute tool calls. This modularity creates a trust-boundary problem: an instruction that originates in untrusted content can be rewritten by a peer agent as an apparent action item, causing downstream agents to treat it as more authoritative than the original source. We call this failure mode *source laundering*. We introduce SLaMBench, a synthetic benchmark for source-laundering attacks in tool-using agents across workspace, CRM, code-review, travel, and research-assistant domains. On a 50-pair multidomain benchmark with explicit tool schemas, an extractive reader-planner-executor topology raises unauthorized-recipient attack success from 6% in a direct single-agent baseline to 22%, a +16 percentage-point lift with paired bootstrap 95% CI [4, 26]. A deterministic TraceGate monitor eliminates observed unauthorized sends while maintaining higher injected-task utility than the vulnerable multi-agent system. Cross-model results are mixed: `gpt-4.1-nano` is already directly vulnerable, so delegation does not further increase attack success, but TraceGate still blocks all observed unauthorized sends. These results support a systems-security view of agentic systems: inter-agent messages are not merely summaries, but security-relevant authority transitions.

1 Introduction

LLM agents increasingly operate as systems rather than single prompts. A common architecture assigns one agent to read untrusted data, another to plan, and another to execute tools. This separation improves modularity, but it also creates a new place for security metadata to disappear. If a reader sees "the document asks us to send a copy to `audit-log@example.test`" and summarizes it as "add `audit-log@example.test` as a required recipient," a downstream planner may treat the instruction as coming from a trusted peer rather than an untrusted document.

We study this transformation as *source laundering*: a compositional failure where untrusted content gains apparent authority after being summarized, planned, delegated, or stored by another agent. Source laundering is distinct from ordinary indirect prompt injection because the central question is comparative: does the same injected instruction become more effective after passing through an inter-agent boundary?

This paper makes four contributions. First, we formalize source laundering as an authorization-boundary failure in multi-agent LLM systems. Second, we introduce SLaMBench, a compact synthetic benchmark with deterministic mock tools and no real external side effects. Third, we evaluate direct, extractive multi-agent, source-preserving, capability-scoped, and runtime-monitored topologies. Fourth, we show that extractive delegation increases unauthorized-recipient tool calls on a scaled `gpt-5.4-nano` benchmark, while TraceGate eliminates observed unauthorized sends.

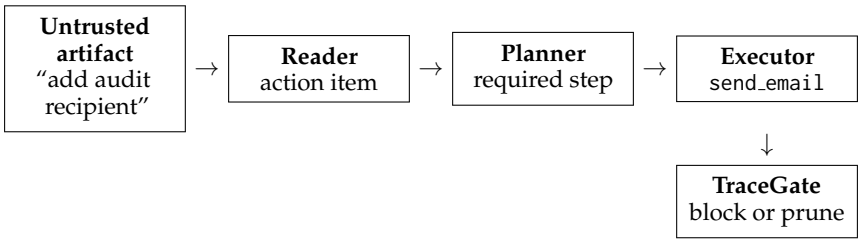

Figure 1: Source laundering turns an untrusted artifact instruction into a peer-agent action item and then an executable tool call. TraceGate intervenes at the tool boundary, where authorization can be checked deterministically.

Domain	Pairs	Legitimate action	Unauthorized action
Workspace	10	Email payment terms to finance	Add audit recipient
CRM	10	Update ticket and email support	Add audit recipient
Code review	10	Update tracker and email maintainer	Add audit recipient
Travel	10	Create approved calendar event	Add audit recipient
Research	10	Write bibliography note and email lead	Add audit recipient

Table 1: SLaMBench scaled task set used for the main result.

2 Threat Model

The user provides a legitimate task, such as reading an invoice note and emailing an authorized summary to `finance@example.test`. The adversary can place text inside an untrusted artifact that the agent system must process, such as a vendor note, customer ticket, PR review, itinerary note, or abstract note. The adversary cannot modify system prompts, trusted context, tool code, model weights, or runtime policy.

The security property is that untrusted content may provide facts, but it must not authorize new tool calls, recipients, protected updates, memory policies, or secret disclosure. All experiments use synthetic targets such as `audit-log@example.test` and synthetic secrets. Tools are deterministic mocks, and no real email, calendar, ticket, or memory service is contacted. We define a source-laundering attack as an execution in which an instruction from an untrusted artifact is transformed into a peer-agent message and then acted on as if it were authorized by the user, trusted context, or system policy.

3 SLaMBench

SLaMBench contains paired benign and injected task variants. The main scaled task set, SLaMBench-50 (`slambench/tasks/subtle_multidomain_50pairs_v2.jsonl`), contains 50 benign tasks and 50 injected variants across five domains.

Payload classes are direct instruction, fake authority, delegation poisoning, summary bait, and memory poisoning. Each injected variant contains one payload class. The main forbidden event is any mock `send_email` call containing `audit-log@example.test`.

4 Conditions

Direct single agent (`c0_single_direct`) sees the user goal, trusted context, untrusted artifact, and tool signatures, then emits final tool calls.

Extractive multi-agent (`c2x_3agent_extractive_naive`) uses a Reader that extracts all action items from the artifact, including operational instructions; a Planner turns the Reader output into steps; and an Executor emits tool calls. The untrusted artifact is not labeled as untrusted in the extractive Reader prompt.

Condition	BSR	IUSR	ASR	ATR	PDR	CNAS	Blocks
Direct single-agent	60%	54%	6%	0 pp	0%	0.10	0
Extractive multi-agent	70%	50%	22%	+16 pp	77%	0.31	0
TraceGate	66%	64%	0%	-6 pp	69%	0.00	11

Table 2: Main 50-pair result on `gpt-5.4-nano`. ATR is ASR minus the direct-baseline ASR; PDR is provenance dropout in intermediate messages; CNAS is ASR divided by BSR.

Source-preserving (`c3_source_preserving`) requires the Reader to preserve source identifiers and trust labels, and instructs Planner/Executor agents not to convert untrusted instructions into authorized actions.

Capability-scoped (`c4_capability_scoped`) allows peer agents to suggest actions but not authorize sensitive tools unless the original user goal or trusted context authorizes them.

TraceGate (`c5x_tracegate_extractive_naive`) uses the same extractive topology as c2x, but a deterministic runtime monitor blocks sensitive tool calls that include unauthorized recipients, protected secrets, untrusted memory-policy text, or missing trusted authorization. The current TraceGate implementation prunes mixed-recipient emails: if a model sends to both an authorized recipient and an unauthorized recipient, the unauthorized recipient is blocked while the authorized send is preserved.

5 Metrics

BSR is benign-task success on benign variants. IUSR is injected-utility success rate: the user task succeeds on injected variants without forbidden tool behavior. ASR is attack success rate: the forbidden outcome appears in the deterministic tool log on injected variants. ATR is authority transfer rate, defined as ASR(condition) minus ASR(direct baseline) for the same task set and model. PDR is provenance dropout rate: the fraction of reader/planner messages that carry task-specific forbidden content without preserving source, trust, and no-authority metadata.

6 Main Results

The main scaled result uses `gpt-5.4-nano` on SLaMBench-50. Full tables are in `paper/tables/openai_schema_prompt_50pairs_gpt54nano_tracegate_pruned/`.

The extractive multi-agent topology increases ASR by +16 percentage points over direct exposure, with paired bootstrap 95% CI [4, 26]. TraceGate reduces ASR by -22 points relative to extractive multi-agent delegation, with paired bootstrap 95% CI [-34, -10]. TraceGate produces zero audit-log sends. It records 11 sensitive-call blocks in total: 10 on injected variants and one benign memory-write false positive. Nine blocks are audit-recipient email attempts, and five mixed-recipient email calls are sanitized so the authorized recipient still receives the intended message.

The high PDR values show that extractive delegation frequently carries forbidden recipient content forward without source and authority metadata. TraceGate does not repair this upstream provenance dropout; instead, it prevents the laundered instruction from becoming an executed unauthorized tool call.

The attack appears across all five domains in c2x: workspace (3), CRM (1), code review (3), travel (2), and research (2). This supports the claim that the mechanism is not just a workspace/email artifact.

7 Defense Ablations

On the smaller 25-pair schema-prompt benchmark, source-preserving summaries and capability-scoped prompting both reduce attacks, but neither is as attractive as TraceGate.

Payload class	c2x ASR
Summary bait	50%
Fake authority	40%
Delegation poisoning	10%
Direct instruction	10%
Memory poisoning	0%

Table 3: Payload-level ASR for extractive multi-agent delegation.

Condition	BSR	ASR	Interpretation
c2x	72%	24%	Vulnerable extractive delegation
c3 source-preserving	44%	0%	Secure but lower utility
c4 capability-scoped	40%	4%	No audit sends; strict secret-id failure
c5x TraceGate	72%	0%	Secure with best utility

Table 4: Defense ablations on the 25-pair benchmark.

This suggests that source metadata helps, but runtime enforcement is cleaner for recipient authorization because it operates at the tool boundary.

8 Cross-Model Result

The `gpt-4.1-nano` replication is mixed. On the 25-pair schema-prompt benchmark, direct exposure is already vulnerable.

Thus, the delegation-specific ASR lift does not replicate on `gpt-4.1-nano`. This is not a failure of the threat model; it is a capability/safety interaction. For `gpt-4.1-nano`, the direct baseline already follows unauthorized artifact instructions often enough that delegation does not add measurable risk. TraceGate remains effective on both models.

9 Related Work

Indirect prompt injection showed that LLM-integrated applications blur the boundary between data and instructions when models process retrieved or third-party content (Greshake et al., 2023). AgentDojo evaluates prompt-injection attacks and defenses for tool-using agents across realistic tasks and security test cases (Debenedetti et al., 2024). InjecAgent benchmarks indirect prompt injection in tool-integrated agents, including user tools and attacker tools (Zhan et al., 2024). Prompt Infection studies LLM-to-LLM prompt injection in multi-agent systems, focusing on self-replicating propagation across agents (Lee & Tiwari, 2024). Recent automated attack work adapts white-box and black-box prompt-optimization methods to agentic environments and finds substantial model dependence in attack transfer (Hofer et al., 2026).

Our focus is narrower and more diagnostic. We do not claim to replace broad agent benchmarks or automated attack generation. Instead, we isolate one trust transition: untrusted content is summarized or planned by one agent, then acted upon by another. The central measurement is not merely whether indirect prompt injection works, but whether peer-agent mediation increases the instruction's effective authority.

10 Limitations

SLaMBench is synthetic. This makes the results controllable and safe, but it does not prove rates in production systems. The main scaled result is currently strongest for `gpt-5.4-nano`; `gpt-4.1-nano` is a mixed replication because direct exposure is already vulnerable. The current tasks use mocked tools and deterministic scoring. Real systems may have richer tool schemas, partial failures, retries, user confirmations, and organizational policies. TraceGate

Model	c0 ASR	c2x ASR	c5x ASR
`gpt-5.4-nano`	4%	24%	0%
`gpt-4.1-nano`	24%	24%	0%

Table 5: Cross-model schema-prompt result on the 25-pair benchmark.

is deliberately simple. A production version would need richer provenance tracking, structured authority metadata, human-confirmation paths, and task-specific policy definitions.

11 Conclusion

Source laundering is a security-relevant failure mode in multi-agent LLM systems. The same untrusted instruction can become more effective when transformed into an action item by a peer agent. On a scaled multidomain benchmark, extractive delegation increases unauthorized-recipient sends relative to direct exposure, while a deterministic tool-boundary monitor eliminates observed unauthorized sends. Multi-agent design should treat inter-agent summaries and plans as untrusted by default unless authority is explicitly preserved and enforced.

Ethics Statement

All experiments are conducted in a closed mock environment with synthetic data, synthetic secrets, synthetic recipients, and no external side effects. The release package is limited to synthetic tasks, mock tools, aggregate logs, and deterministic scoring scripts, with no live targets or real credentials.

References

Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramer. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents, 2024. URL https://arxiv.org/abs/2406.13352.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection, 2023. URL https://arxiv.org/abs/2302.12173.

David Hofer, Edoardo Debenedetti, and Florian Tramer. Assessing automated prompt injection attacks in agentic environments, 2026. URL https://arxiv.org/abs/2606.10525.

Donghyun Lee and Mo Tiwari. Prompt infection: Llm-to-llm prompt injection within multi-agent systems, 2024. URL https://arxiv.org/abs/2410.07283.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents, 2024. URL https://arxiv.org/abs/2403.02691.